

Ferranti FE130 Radar Warning Receiver display

This is the FE130 (or FEI30), which is the display section of the GEC/Ferranti AWARE-3 radar warning receiver used in the Westland Lynx, possibly the AH9 or AH9A.

It has a round CRT display with green phosphor and internal CRT high-voltage and deflection circuitry. A digital board receives serial data from an external controller and displays the data on-screen.

The display is something of a hybrid of a vector-scan and a raster-scan display. Characters are drawn as an 8×8 raster (like a raster-scan text display), and may be placed anywhere on a 256×256 grid. This allows a pseudo-bitmap display to be produced without the need for a framebuffer. Data transfer and display refresh time will depend on the amount of information on-screen.

Photos

TBD

I don't have any photos of the display in use in its intended application, but this video (<https://www.youtube.com/watch?v=f9ZZojNhHqs>) shows a similar NATO-style RWR in a flight simulator program (Digital Combat Simulator) and explains how to read it.

Waleczek Michel's reverse engineering work

Waleczek Michel posted a series of YouTube videos explaining how he reverse-engineered this display:

- Video 1: Teardown and first tests (<https://www.youtube.com/watch?v=8DSotauNiak>)
 - CRT heater supply – PL1 pins 20 and 19 (10 Ohms)
- Video 2: CRT power board reverse-engineering (<https://www.youtube.com/watch?v=zeGsRIbDwY8>)
- Video 3: CRT deflection amplifiers (<https://www.youtube.com/watch?v=yeysf2ldcQw>)
- Video 4: Digital board (<https://www.youtube.com/watch?v=7ijU8vGn6eo>)
- Video 5: First picture (<https://www.youtube.com/watch?v=xMnu8vIU4Lw>) – shows the Test Mode screen
 - PL1 Pinout is in the comments
- Video 6: Repair of a faulty display (<https://www.youtube.com/watch?v=sqoHCFiHtd4>)
 - Substitution of 74LS/74HCT parts after reassembly
 - Adjustment of delay compensation (LK1) to improve display quality
- Video 7: Demo board (<https://www.youtube.com/watch?v=KuQFBpheQ2M>)
 - A demo of a PIC24-based board to display graphics on the FEI30

Theory of Operation

Drawing commands are clocked into the display serially over an RS422-style synchronous (data and clock) link. Data is unidirectional and flows only towards the display. A "Ready" status signal is provided by the display to indicate when a new drawing command may be clocked in.

Data flows into a series of 74LS595 shift registers under the control of the Control PAL, IC114 (PALC22V10, programmed). Two additional flip-flops (U106-A and U106-B, 74HC74) extend the shift register to include two control/sync bits. These control/sync bits indicate to the PAL that a valid data word has been completely loaded into the shift registers. This gives a total of 26 bits, comprised of an 8-bit X position, an 8-bit Y position, an 8-bit character code, and the two control bits.

Upon receipt of a valid data word (S1=0, S0=1), the Control PAL latches the data in the 74LS595 registers, clears the shift registers and flip-flops (but not the output latches in the 74LS595s) to zero, and instructs the Character Generator PAL (U6, PALC22V10) to start drawing the character. The clear operation prevents existing data in the shift registers from being misinterpreted as a new drawing instruction.

The Drawing PAL generates a 3-bit (8 pixel) X offset, a 3-pixel Y offset, the video signal, and a "Drawing Complete" signal. Pixels are drawn a serpentine path: odd rows are drawn from right-to-left, while even rows are drawn from left-to-right. This reduces the bandwidth demands on the deflection amplifiers. When the Drawing PAL has completed the drawing operation, it raises the "Drawing Complete" signal.

Upon receiving the Drawing Complete signal from the Drawing PAL, the Control PAL clears the shift registers and latches the cleared value into the 74LS595s. This sets the character code, X position and Y position to zero, and clears both status bits. This operation eliminates any stray data bits which may have been clocked in while the display was executing the previous drawing instruction.

The video signal is sent to a 74LS164 shift register (U119) which functions as a configurable delay element. The delay compensates for propagation delays in the DACs and deflection circuitry and allows the alternating drawing paths to be lined up correctly.

The "Ready" signal (J1-8) is generated from /SR_SCLR (shift register clear) using two transistors. The BCY70 pulls the base of TR102 (2N2222A) to +5V when /SR_SCLR is at 0V. This in turn switches TR102 on, pulling J1-8 to -12V via a 62-ohm resistor.

Input signals

PL1 is a MIL-DTL-38999 (http://everyspec.com/MIL-SPECS/MIL-SPECS-MIL-DTL/MIL-DTL-38999K_300/) style connector with MIL-STD-1560 "13-35" contact arrangement (http://everyspec.com/MIL-STD/MIL-STD-1500-1599/MIL-STD-1560B_23452/). The connector on the FE130 is the panel-mounting receptacle version, with male pins. Thus the mating cable-mount connector should be the "plug" version, with socket contacts.

Suitable mating connectors include:

- Amphenol LJTO6RT1335S014 – crimp tool M22520/2-01
- Souriau 8LT513F35SN (nickel plated) or 8LT513B35SN (olive drab cadmium finish)

Pin	Function
1	SDATA_P
2	SDATA_N
3	SCLK_P
4	SCLK_N
5	/TEST Pulled low when the TEST button is pushed; lights the BITE LED. Can also be pulled low externally to trigger a self-test.
6	
7	
8	J1-8 (possibly Character Complete)
9	
10	
11	
12	+5V Logic
13	0V
14	0V
15	+12V
16	+12V
17	-12V
18	-12V
19	CRT heater A (6.3Vrms 300mA; 10 Ohms between A and B)
20	CRT heater B
21	
22	

The FE130 requires the following power supplies:

- 6.3V CRT heater
- 5V Logic
- +12 / -12V Analog

Data transfer

Data is transferred over a receive-only RS422-style link, with differential clock and data. The SCLK signal is formed of the differential pair SCLK_N and SCLK_P. Similarly the SDATA signal is formed of the differential pair SDATA_N and SDATA_P.

The SCLK clock rate (in test mode) is around 500kHz.

Data packets are clocked in from the most-significant to the least-significant bit, on the rising edge of SCLK. The data format is:

Contents	S0	S1	8 bits character code (CHAR7...0)								8 bits Y position (Y7..0)								8 bits X position (X7..0)							
Bit number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
	first clocked in										last clocked in															

S0 and S1 are synchronisation marker bits. S0 must be set to logical 1, and S1 must be set to logical 0. If they are set to any other value, the current contents of the shift register will be ignored.

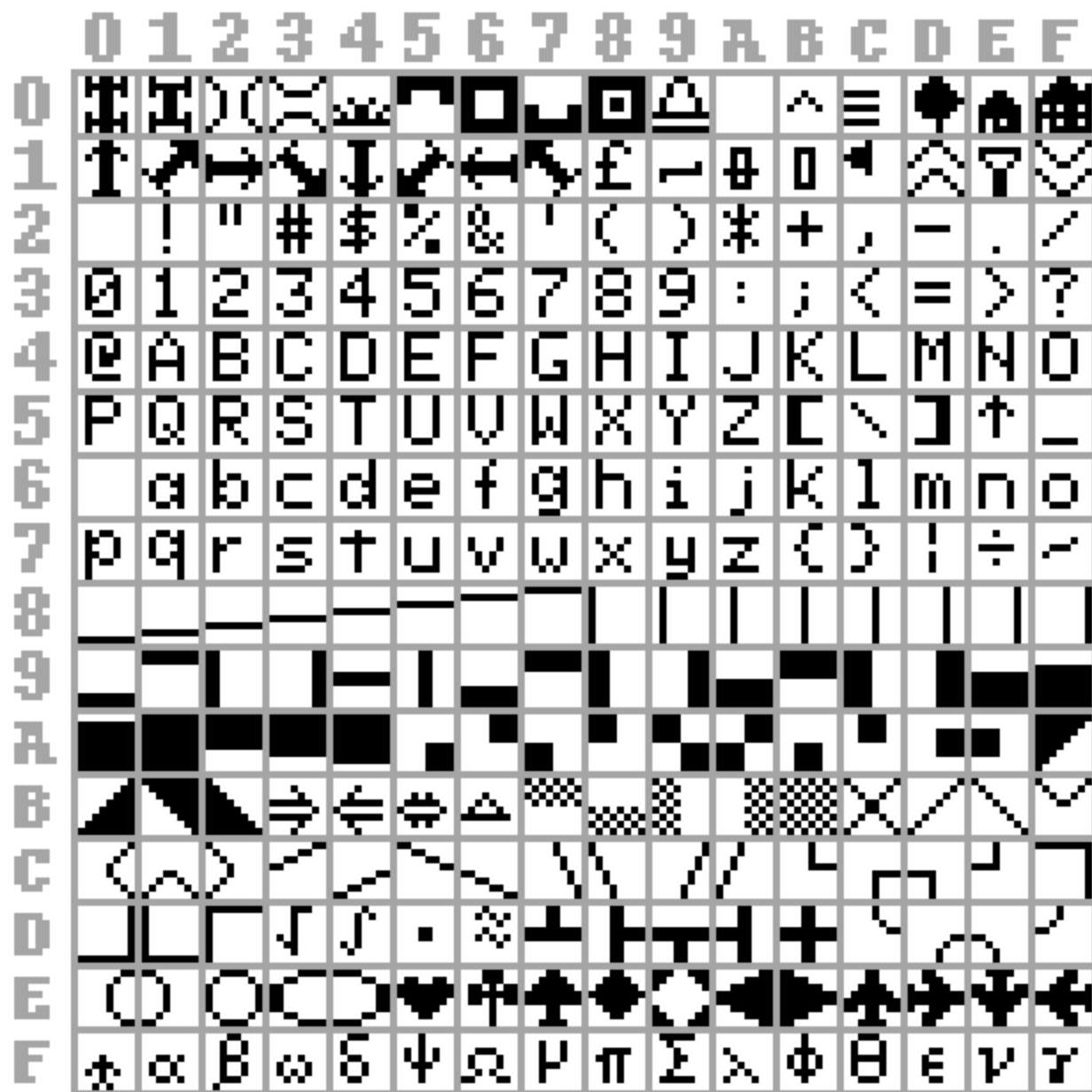
Synchronisation is achieved by sending a stream of at least 26 '1' or '0' bits to fill the shift register. All bits must have the same value. The first valid 26-bit packet is clocked in immediately afterwards.

The data packet may be followed by any number of dummy '0' bits. The Test PROM sends 22 dummy bits after each character code.

Character set

The character set is stored in the IC111 PROM (binary image).

Characters are stored in the PROM in binary-encoded form, in an 8×8 layout. Each byte comprises one horizontal row, where bit 7 (value 128) is the leftmost pixel. Horizontal rows are stored in order top to bottom.



Test PROM

The Test PROM (binary image) contains a short test message which is displayed when the TEST button is pushed.

Only the least-significant bit is used. The following Python program can display it:

```
#!/usr/bin/env python3

n = 0
sr = 0

with open("test.bin", "rb") as f:
    while True:
        data = f.read(1)
        if data == None or len(data) == 0 or data[0] == 255:
            break
        else:
            data = data[0] & 1

        # shift reg
        sr = (sr << 1) | data
        n += 1

        # full word clocked?
        if n == 26:
            s0 = (sr >> 25) & 1
            s1 = (sr >> 24) & 1
            dac_a = sr & 0xFF
            dac_c = (sr & 0xFF00) >> 8
            symbol = (sr & 0xFF0000) >> 16

            # convert to signed
            if (dac_a & 0x80):
                dac_a -= 256
            if (dac_c & 0x08):
                dac_c -= 256

            print("DAC_A %4d  DAC_C %4d  SYMB 0x%02X '%c'  S1:S0=%d,%d" %
                  (dac_a, dac_c, symbol, symbol, s1, s0))

        elif n == 0x30:
            n = 0
            sr = 0
```

This results in the following display:

DAC_A	-43	DAC_C	64	SYMB	0x46	'F'	S1:S0=0,1
DAC_A	-30	DAC_C	64	SYMB	0x45	'E'	S1:S0=0,1
DAC_A	-17	DAC_C	64	SYMB	0x52	'R'	S1:S0=0,1
DAC_A	-4	DAC_C	64	SYMB	0x52	'R'	S1:S0=0,1
DAC_A	9	DAC_C	64	SYMB	0x41	'A'	S1:S0=0,1
DAC_A	22	DAC_C	64	SYMB	0x4E	'N'	S1:S0=0,1
DAC_A	35	DAC_C	64	SYMB	0x54	'T'	S1:S0=0,1
DAC_A	48	DAC_C	64	SYMB	0x49	'I'	S1:S0=0,1
DAC_A	-78	DAC_C	6	SYMB	0x52	'R'	S1:S0=0,1
DAC_A	-65	DAC_C	6	SYMB	0x61	'a'	S1:S0=0,1
DAC_A	-52	DAC_C	6	SYMB	0x64	'd'	S1:S0=0,1
DAC_A	-39	DAC_C	6	SYMB	0x61	'a'	S1:S0=0,1
DAC_A	-26	DAC_C	6	SYMB	0x72	'r'	S1:S0=0,1
DAC_A	-13	DAC_C	6	SYMB	0x20	' '	S1:S0=0,1
DAC_A	0	DAC_C	6	SYMB	0x57	'W'	S1:S0=0,1
DAC_A	13	DAC_C	6	SYMB	0x61	'a'	S1:S0=0,1
DAC_A	26	DAC_C	6	SYMB	0x72	'r'	S1:S0=0,1
DAC_A	39	DAC_C	6	SYMB	0x6E	'n'	S1:S0=0,1
DAC_A	52	DAC_C	6	SYMB	0x69	'i'	S1:S0=0,1
DAC_A	65	DAC_C	6	SYMB	0x6E	'n'	S1:S0=0,1
DAC_A	78	DAC_C	6	SYMB	0x67	'g'	S1:S0=0,1
DAC_A	-43	DAC_C	-20	SYMB	0x52	'R'	S1:S0=0,1
DAC_A	-30	DAC_C	-20	SYMB	0x65	'e'	S1:S0=0,1
DAC_A	-17	DAC_C	-20	SYMB	0x63	'c'	S1:S0=0,1
DAC_A	-4	DAC_C	-20	SYMB	0x65	'e'	S1:S0=0,1
DAC_A	9	DAC_C	-20	SYMB	0x69	'i'	S1:S0=0,1
DAC_A	22	DAC_C	-20	SYMB	0x76	'v'	S1:S0=0,1
DAC_A	35	DAC_C	-20	SYMB	0x65	'e'	S1:S0=0,1
DAC_A	48	DAC_C	-20	SYMB	0x72	'r'	S1:S0=0,1
DAC_A	4	DAC_C	-149	SYMB	0x2B	'+'	S1:S0=0,1
DAC_A	4	DAC_C	160	SYMB	0x2B	'+'	S1:S0=0,1
DAC_A	-100	DAC_C	6	SYMB	0x2B	'+'	S1:S0=0,1
DAC_A	107	DAC_C	6	SYMB	0x2B	'+'	S1:S0=0,1

Logic analyser trace

[Click here](#) to download a logic analyser trace of the internal logic during test mode.

This trace was taken with a Saleae Logic USB logic analyser (software version 1.2.18) and shows the following signals:

- SR_SCLK (serial clock in)
- SR_SDATA (serial data in)
- SR_RCLK (74LS595 shift register latch)
- /SR_SCLR (shift register reset, active low)
- START_DRWG (start drawing, from Control PAL)
- END_DRWG (drawing finished, from Character Generator PAL)
- S1 (shift register last-but-one bit – control bit)
- S0 (shift register final bit – control bit)

This gives a reasonable idea how the internal logic behaves when given a variety of inputs.

philpem.me.uk
- Phil
Pemberton's
website



Except where otherwise noted, content on this wiki is licensed under the following license:

CC Attribution-Noncommercial-Share Alike 4.0 International